Лекция 4. Первичные ошибки, вторичные ошибки и их проявления.

В предыдущих лекциях мы подробно разобрали, что такое объекты уязвимости, дестабилизирующие факторы и угрозы надёжности. Мы выяснили, что любая компьютерная система потенциально уязвима, и что угрозы могут реализоваться через конкретные слабые места под воздействием различных факторов. Мы также рассмотрели методы предотвращения угроз надёжности и виды избыточности, которые позволяют системе сохранять работоспособность даже при возникновении отказов и сбоев.

Однако для полноценного понимания надёжности необходимо разобраться ещё в одном важном вопросе: как именно возникают ошибки? Почему одна ошибка может привести к целой цепочке последствий? И почему иногда система, в которой «всё правильно», всё равно даёт сбои?

Ответ на эти вопросы лежит в области теории ошибок. Ошибки не возникают изолированно – они образуют цепочки, каскады и взаимосвязи. Одна ошибка может породить другую, та – третью, и в итоге система приходит к состоянию, которое никто не планировал. В данной лекции мы рассмотрим, что такое первичные ошибки и как они возникают, что такое вторичные ошибки и чем они отличаются от первичных, как ошибки проявляются на разных уровнях системы, как классифицировать ошибки по причинам и последствиям, а также как предотвращать цепочки ошибок. Материал этой лекции является фундаментом для понимания того, почему системы отказывают даже при наличии защитных механизмов, и как правильно анализировать причины инцидентов.

1. Понятие ошибки

Ошибка – это неправильный результат работы системы, который может быть вызван как внутренними, так и внешними причинами. Ошибка может не приводить к отказу, но снижает качество функционирования. Важно понимать, что ошибка – это не всегда «поломка». Это может быть:

- неверный результат вычисления;

- неправильное отображение данных;

- потеря информации;

- несанкционированный доступ;

- задержка в обработке;

- некорректная реакция на действие пользователя.

1.2. Ошибка, дефект, отказ – в чём разница

В теории надёжности и тестировании ПО часто путают три понятия: дефект, ошибка и отказ.

- Дефект (defect) – недостаток в коде, документации или конфигурации. Пример: ошибка в формуле расчёта.

- Ошибка (error) – неправильный результат работы системы. Пример: баланс посчитан неверно.

- Отказ (failure) – событие, при котором система теряет работоспособность. Пример: программа упала.

Эти три понятия образуют цепочку: дефект → ошибка → отказ.

1.3. Почему ошибки образуют цепочки

Ошибки не существуют изолированно. **Одна ошибка может стать причиной другой, та – третьей, и так далее. Это называется каскадом ошибок (error cascade).**

Рассмотрим пример:

1. В коде есть дефект – не обрабатывается деление на ноль.

2. Пользователь вводит ноль – возникает ошибка (исключение).

3. Программа не обрабатывает исключение – происходит отказ (падение).

4. Отказ приводит к потере данных – ещё одна ошибка.

5. Потеря данных приводит к неверной отчётности – третья ошибка.

2. Первичные ошибки

2.1. Что такое первичная ошибка

Первичная ошибка (primary error) – это ошибка, которая возникает непосредственно под воздействием дестабилизирующего фактора на объект уязвимости. Это первое звено в цепочке ошибок. Первичная ошибка – это корневая причина (root cause) инцидента. Если её устранить, вся цепочка последующих ошибок не возникнет.

2.2. Причины первичных ошибок

Первичные ошибки могут возникать по разным причинам:

- **Ошибки проектирования:** неверно выбрана архитектура, не учтены требования, неправильно определены границы системы, не предусмотрены механизмы защиты.

- **Ошибки разработки**: синтаксические ошибки, логические ошибки, ошибки в алгоритмах, необработанные исключения, ошибки в работе с памятью.

- **Ошибки конфигурирования**: неверные настройки, слабые пароли, открытые порты, отсутствие разграничения прав.

- **Ошибки ввода данных:** опечатки, неверный формат, пропущенные значения, некорректные единицы измерения.

- **Аппаратные ошибки:** производственный брак, износ, перегрев, скачки напряжения.

2.3. **Как обнаружить первичную ошибку**

Обнаружение первичной ошибки – задача непростая, потому что она может быть скрыта за множеством вторичных проявлений. **Основные методы**:

- **Анализ корневых причин** (Root Cause Analysis, RCA) **– метод, при котором последовательно задаётся вопрос «почему?» до тех пор, пока не будет найдена истинная причина**.

- **Логирование – запись всех действий и событий системы.**

- **Трассировка – отслеживание пути выполнения программы.**

- **Тестирование – выявление дефектов на ранних этапах.**

- **Мониторинг – постоянное наблюдение за состоянием системы**.

3. Вторичные ошибки

3.1. Что такое вторичная ошибка

Вторичная ошибка (secondary error) – это ошибка, которая возникает как следствие первичной ошибки. Она не является самостоятельной причиной инцидента, а представляет собой реакцию системы на первичную ошибку. Вторичные ошибки – это проявления первичной ошибки на разных уровнях системы.

3.2. Как вторичные ошибки связаны с первичными

Вторичные ошибки могут быть:

- Прямыми следствиями – возникают сразу после первичной ошибки. Пример: первичная ошибка – деление на ноль; вторичная ошибка – исключение в программе.

- Косвенными следствиями – возникают через некоторое время. Пример: первичная ошибка – перегрев процессора; вторичная ошибка – снижение производительности через час.

- Каскадными следствиями – порождают новые ошибки. Пример: первичная ошибка – сбой питания; вторичная ошибка – повреждение файла; третичная ошибка – невозможность запуска программы.

3.3. Почему вторичные ошибки опасны

Вторичные ошибки опасны по нескольким причинам:

1. Маскируют первичную ошибку. Разработчик видит вторичную ошибку и пытается исправить её, не понимая, что причина – в другом месте.

2. Усиливают последствия. Каждая новая ошибка в цепочке усугубляет ситуацию.

3. Распространяются по системе. Одна ошибка может затронуть множество модулей.

4. Трудно диагностируются. Вторичная ошибка может проявляться далеко от места первичной.

5. Приводят к новым ошибкам. Каскад может расти бесконечно.

4. Проявления ошибок

4.1. Уровни проявления ошибок

Ошибки могут проявляться на разных уровнях системы:

- Аппаратный уровень: отказ компонента, перегрев, потеря питания, ошибки памяти.

- Системный уровень: сбой операционной системы, ошибки драйверов, конфликты ресурсов, зависание процессов.

- Прикладной уровень: ошибки в бизнес-логике, необработанные исключения, неверные расчёты, потеря данных.

- Пользовательский уровень: неправильное отображение, некорректные подсказки, зависание интерфейса, потеря введённых данных.

- Организационный уровень: нарушение бизнес-процессов, финансовые потери, репутационный ущерб, юридические последствия.

4.2. Формы проявления ошибок

Ошибки могут проявляться в разных формах:

- Явные (немедленные): программа падает, появляется сообщение об ошибке, система зависает, данные не сохраняются.

- Скрытые (отложенные): неверные данные накапливаются, производительность постепенно снижается, ошибка проявляется только при определённых условиях, проблема обнаруживается через дни или недели.

- Плавающие: ошибка возникает периодически, зависит от нагрузки, зависит от времени, трудно воспроизводится.

4.3. Симптомы и причины

Важно различать симптомы и причины:

- Симптом – то, что мы наблюдаем (программа падает).

- Причина – то, что вызывает симптом (деление на ноль).

Симптомы могут быть обманчивыми. Один и тот же симптом может быть вызван разными причинами, и одна и та же причина может проявляться разными симптомами.

- Симптом: «программа работает медленно».

- Причины: перегрев процессора, утечка памяти, вирус, фрагментация диска.

5. **Классификация ошибок**

Ошибки можно классифицировать по нескольким основаниям, и каждая классификация помогает увидеть проблему с определённой стороны. Это важно для диагностики, поскольку одна и та же ошибка может быть описана по-разному в зависимости от того, что именно мы хотим подчеркнуть – её причину, время обнаружения, серьёзность или частоту появления.

По причинам возникновения выделяют несколько типов ошибок. **Проектные ошибки** связаны с недостатками архитектуры или неверными решениями на этапе проектирования – например, неверно спроектированная схема базы данных, которая не выдерживает нагрузку. **Кодовые ошибки** возникают из-за дефектов в исходном коде: необработанное исключение, неверная формула, ошибка в логическом условии. **Конфигурационные ошибки** появляются из-за неправильных настроек – открытый порт, слабый пароль, отсутствие разграничения прав доступа. **Ошибки данных** связаны с некорректными входными значениями: неверный формат, пропущенное поле, опечатка. **Аппаратные ошибки** возникают из-за отказов оборудования – сбой диска, перегрев процессора, выход из строя блока питания. И, наконец, **человеческие ошибки** связаны с действиями пользователя: случайное удаление файла, неверный ввод, нарушение инструкции. **Понимание причины ошибки – первый шаг к её устранению, потому что меры профилактики для каждого типа будут разными.**

По времени обнаружения ошибки делятся на ранние, средние и поздние. **Ранние ошибки** выявляются на этапе разработки – это самый благоприятный сценарий, потому что исправление стоит дёшево и не влияет на пользователей. **Средние** обнаруживаются на этапе тестирования – это тоже приемлемо, хотя требует дополнительных затрат времени и ресурсов. **Поздние ошибки** проявляются уже на этапе эксплуатации – они самые дорогие, потому что могут привести к потерям данных, финансовым убыткам и репутационному ущербу. Чем раньше обнаружена ошибка, тем меньше последствий она вызовет.

По степени серьёзности ошибки делятся на критические, высокие, средние и низкие. **Критическая ошибка** приводит к тому, что система полностью перестаёт работать или происходит потеря данных – например, падение сервера или повреждение базы данных. **Высокая ошибка** означает, что функция не работает, но система в целом продолжает функционировать – например, не работает экспорт отчёта, но остальные операции доступны. **Средняя ошибка** – функция работает, но с отклонениями, которые не блокируют работу, но снижают удобство или точность. **Низкая ошибка** – это косметические дефекты, которые не влияют на функциональность, но портят впечатление от системы. Классификация по серьёзности помогает расставить приоритеты при исправлении: сначала критические, потом высокие и так далее.

По повторяемости ошибки бывают систематическими, случайными и единичными. **Систематическая ошибка** возникает при каждом выполнении определённого сценария – например, при вводе отрицательного числа программа всегда падает. **Случайная ошибка** возникает периодически, часто зависит от нагрузки, времени или стечения обстоятельств – её сложнее воспроизвести и диагностировать. **Единичная ошибка** возникает однократно и может быть следствием уникального стечения условий – например, сбой из-за редкого сочетания факторов. Понимание повторяемости помогает выбрать стратегию тестирования и отладки.

Все эти классификации не существуют отдельно друг от друга – одна и та же ошибка может быть одновременно, например, кодовой, поздней, критической и систематической. Комплексный взгляд на ошибку с разных сторон позволяет точнее определить её природу, выбрать правильные меры устранения и предотвратить её появление в будущем.

6. Цепочки ошибок и их анализ

6.1. Модель цепочки ошибок

Для анализа ошибок удобно использовать следующую модель:

Дестабилизирующий фактор → Первичная ошибка → Вторичная ошибка → Третичная ошибка → Последствие

Пример:

1. Фактор: скачок напряжения.

2. Первичная ошибка: выход из строя блока питания.

3. Вторичная ошибка: отказ сервера.

4. Третичная ошибка: потеря данных.

5. Последствие: недоступность системы, финансовые потери.

6.2. Метод «5 почему»

Метод «5 почему» (5 Whys) – простой, но эффективный инструмент анализа корневых причин. Суть метода: последовательно задавать вопрос «почему?» до тех пор, пока не будет найдена истинная причина.

Пример:

1. Почему система упала? – Произошло исключение.

2. Почему произошло исключение? – Деление на ноль.

3. Почему произошло деление на ноль? – Пользователь ввёл ноль.

4. Почему пользователь ввёл ноль? – Нет проверки ввода.

5. Почему нет проверки ввода? – Забыли добавить в код.

Корневая причина: отсутствие проверки ввода.

6.3. **Анализ корневых причин (RCA)**

Root Cause Analysis (RCA) – более формальный метод, который включает:

1. Сбор данных об инциденте.

2. Определение последовательности событий.

3. Выявление первичной ошибки.

4. Анализ причин её возникновения.

5. Разработка корректирующих мер.

6. Внедрение и проверка.

7. Предотвращение цепочек ошибок

7.1. Общие принципы

Для предотвращения цепочек ошибок необходимо:

1. Устранять первичные ошибки – бороться с корневой причиной.

2. Разрывать цепочки – не давать ошибке распространяться.

3. Обнаруживать ошибки как можно раньше – на этапе разработки и тестирования.

4. Изолировать ошибки – ограничивать область их влияния.

5. Восстанавливать систему – возвращать её в работоспособное состояние.

7.2. Методы предотвращения

Предотвращение ошибок – это не единичное действие, а непрерывный процесс, который охватывает все этапы жизненного цикла программного обеспечения. На каждом этапе существуют свои методы, которые помогают выявить и устранить ошибки до того, как они приведут к серьёзным последствиям. Рассмотрим эти методы последовательно.

На этапе проектирования закладывается фундамент будущей системы, и именно здесь дешевле всего предотвратить ошибки. **Анализ рисков** позволяет заранее выявить потенциально опасные места и предусмотреть меры защиты. **Проектирование с учётом отказоустойчивости** означает, что архитектура системы изначально строится так, чтобы отказ одного компонента не приводил к отказу всей системы. **Выбор проверенных технологий** снижает вероятность появления ошибок, связанных с незрелыми или плохо документированными инструментами. **Документирование** помогает зафиксировать принятые решения и избежать разночтений в будущем.

На этапе разработки основная задача – не допустить появления ошибок в коде и как можно раньше их обнаружить. Код-ревью, то есть взаимная проверка кода разработчиками, позволяет выявить ошибки, которые не заметил автор. **Статический анализ кода** автоматически проверяет программу на наличие потенциальных проблем – неиспользуемых переменных, необработанных исключений, утечек памяти. **Модульное тестирование** проверяет отдельные функции и модули на корректность работы. **Обработка исключений** гарантирует, что программа не упадёт при возникновении непредвиденной ситуации, а корректно отреагирует на неё.

На этапе тестирования система проверяется в условиях, приближенных к реальной эксплуатации. **Функциональное тестирование** проверяет, что все заявленные функции работают так, как описано в требованиях. **Нагрузочное тестирование** показывает, как система ведёт себя при высокой нагрузке – например, при одновременной работе тысячи пользователей. **Тестирование на отказ** имитирует отказы отдельных компонентов и проверяет, способна ли система сохранить работоспособность. **Тестирование безопасности** выявляет уязвимости, которые могут быть использованы злоумышленниками.

На этапе эксплуатации основная задача – как можно раньше обнаружить возникающие проблемы и минимизировать их последствия. **Мониторинг** позволяет постоянно отслеживать состояние системы и замечать отклонения от нормальной работы. **Логирование** фиксирует все значимые события, что помогает при анализе инцидентов. **Резервное копирование** гарантирует, что данные можно будет восстановить в случае потери. **Обновление** позволяет устранять обнаруженные уязвимости и улучшать работу системы.

Все эти методы работают в комплексе: если на одном этапе что-то было упущено, следующий этап может это компенсировать. Однако, чем раньше обнаружена ошибка, тем дешевле её исправление и тем меньше последствий она вызовет. Поэтому основная стратегия предотвращения – это смещение усилий в сторону ранних этапов: лучше потратить время на проектирование и код-ревью, чем потом разбираться с последствиями инцидентов в эксплуатации.

7.3. Роль избыточности

Виды избыточности, рассмотренные в предыдущей лекции, играют ключевую роль в предотвращении цепочек ошибок:

- Структурная – дублирование компонентов, чтобы отказ одного не приводил к отказу системы.

- Информационная – контрольные суммы, чтобы обнаружить искажение данных.

- Программная – независимые версии модулей, чтобы ошибка в одной не повлияла на результат.

- Временная – контрольные точки, чтобы восстановиться после сбоя.

- Функциональная – перераспределение задач, чтобы отказ одного компонента компенсировался другим.

8. Практическое задание

Задание: «Анализ цепочки ошибок для конкретной системы»

Исходные данные (выберите один вариант):

- Вариант А: Интернет-магазин, обслуживающий 10 000 пользователей в день.

- Вариант Б: Бухгалтерская система, хранящая финансовые документы.

- Вариант В: Медицинская информационная система (МИС) для поликлиники.

Ход выполнения:

Шаг 1. Опишите инцидент для выбранной системы. Укажите:

- что произошло (симптом);

- как это проявилось (форма проявления);

- какие последствия.

Шаг 2. Для инцидента постройте цепочку ошибок в формате этап-описание:

1. Дестабилизирующий фактор – описание

2. Первичная ошибка – описание

3. Вторичная ошибка – описание

4. Третичная ошибка – описание

5. Последствие – описание.

Шаг 3. Для первичной ошибки примените метод «5 почему» и определите корневую причину.

Шаг 4. Предложите меры предотвращения для первичной ошибки. Укажите:

- тип меры (техническая, организационная, программная);

- какой вид избыточности поможет;

- как мера разорвёт цепочку ошибок.

Пример выполнения практического задания

Вариант: Система управления лифтом в многоэтажном бизнес-центре

Краткое описание системы: Система управления лифтом – это встроенная вычислительная система, которая управляет движением кабины, открытием и закрытием дверей, реагирует на вызовы пассажиров, обеспечивает безопасность и взаимодействует с диспетчерской. Система работает круглосуточно, от её надёжности зависит жизнь и здоровье людей.

Шаг 1. Описание трёх инцидентов

Инцидент №1. Лифт не приезжает на вызов

Пассажир на 7-м этаже нажал кнопку вызова, но лифт не приехал. Через некоторое время выяснилось, что система «потеряла» вызов. Симптом – вызов не обрабатывается. Форма проявления – скрытая (пассажир думает, что лифт занят, и ждёт). Последствия – длительное ожидание, недовольство, снижение пропускной способности лифта.

Шаг 2. Построение цепочек ошибок

Инцидент №1. Лифт не приезжает на вызов

Дестабилизирующий фактор – переполнение очереди вызовов. Первичная ошибка – очередь вызовов не обработала новый вызов. Вторичная ошибка – вызов потерян, система о нём не знает. Третичная ошибка – лифт не приезжает, пассажир ждёт. Последствие – длительное ожидание, снижение пропускной способности.

Шаг 3. Применение метода «5 почему»

Инцидент №1. Лифт не приезжает на вызов

Почему лифт не приехал? Вызов не был обработан. Почему вызов не был обработан? Очередь вызовов переполнена. Почему очередь переполнена? Нет ограничения на количество вызовов. Почему нет ограничения? Не предусмотрено в алгоритме. Почему не предусмотрено? Не проведено тестирование при пиковой нагрузке.

Корневая причина – отсутствие нагрузочного тестирования системы управления вызовами.

Шаг 4. Меры предотвращения

Инцидент №1. Лифт не приезжает на вызов

- Ввести ограничение на размер очереди вызовов. Тип меры – программная. Вид избыточности – информационная. Предотвратит переполнение.

- Добавить резервный контроллер вызовов. Тип меры – техническая. Вид избыточности – структурная. При отказе основного вызовы обработает резервный.

- Провести нагрузочное тестирование. Тип меры – организационная. Выявит проблему до эксплуатации.